

Practical – 3

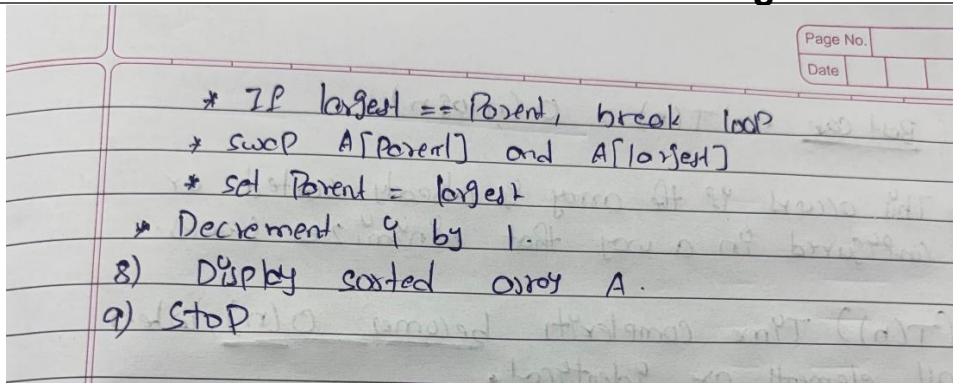
Aim : To Implementation of Max-Heap sort algorithm

Algorithm :

Heap Sort Algorithm

Input : Array A with n elements
Output : Sorted Array A with n in ascending order.

- 1) Start
- 2) Read n and array A
- 3) Set heap_size = n
- 4) Set i = floor(n/2)
- 5) Repeat while i >= 1:
 - * Set Parent = i
 - * Repeat while (2 * Parent) <= heap_size:
 - * Set left = 2 * Parent
 - * Set right = left + 1
 - * Set largest = Parent
 - * IF A[left] > A[largest], set largest = left.
 - * IF right <= heap_size and A[right] > A[largest], set largest = right
 - * IF largest == Parent, break loop
 - * swap A[Parent] and A[largest]
 - * Set Parent = largest
 - * Decrement i by 1.
- 6) Set i = n
- 7) Repeat while i >= 2:
 - * swap A[1] and A[i]
 - * set heap_size = heap_size - 1
 - * set Parent = 1
 - * Repeat while (2 * Parent) <= heap_size:
 - * set left = 2 * Parent
 - * set right = left + 1
 - * set largest = Parent
 - * IF A[left] > A[largest], set largest = left.
 - * IF right <= heap_size and A[right] > A[largest], set largest = right



Program / Code :

```
#include <iostream>
using namespace std;
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}
```

```
}  
int main() {  
    int n;  
    cout << "Enter the number of elements: ";  
    cin >> n;  
    int arr[n];  
    cout << "Enter " << n << " elements:\n";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
    cout << "\nOriginal array: ";  
    for (int i = 0; i < n; i++)  
        cout << arr[i] << " ";  
    heapSort(arr, n);  
    cout << "\nSorted array: ";  
    for (int i = 0; i < n; i++)  
        cout << arr[i] << " ";  
    cout << endl;  
    return 0;  
}
```

Output :

```
C:\Users\NIKSHITH\DAA>g++ "Heap Sort.cpp" -o heapsort
```

```
C:\Users\NIKSHITH\DAA>heapsort
```

```
Enter the number of elements: 5
```

```
Enter 5 elements:
```

```
9
```

```
4
```

```
1
```

```
8
```

```
3
```

```
Original array: 9 4 1 8 3
```

```
Sorted array: 1 3 4 8 9
```

Time Complexity :

Best Case : $O(n \log n)$

Average Case : $O(n \log n)$

Worst Case : $O(n \log n)$

Heap sort Time complexity

heap-size = n ;

for (i = n/2 ; i >= 1 ; i--) { // (total call) → n
} for (i = n ; i >= 2 ; i--) { → n
 swap(A[i], A[1]); → 1
 heap-size = heap-size - 1 ; → 1
}

Max-Heapify (A, i, heap-size); → log n
}

$$T(n) = 1 + n + n \times (1 + 1 + \log n)$$
$$= 2n + 1 + n(2 + \log n)$$
$$= 2n + 1 + 2n + n \log n$$
$$= 4n + 1 + n \log n$$

∴ we take highest order term

$T(n) = O(n \log n)$

Best case : $T(n) = O(n \log n)$

→ This occurs if the array is already sorted or configured in a way that minimizes swaps.

⇒ $T(n)$ Time complexity becomes $O(n)$, when all elements are identical,

Eg: $[7, 7, 7, 7]$

→ the algorithm looks at the Parent node & compares them to their children.

→ Is Parent 7 smaller than child 7? No.

As a Result "Zero" swaps are made.

Average case : $T(n) = O(n \log n)$

→ This occurs when the elements are in completely random, jumbled order.

Worst case : $T(n) = O(n \log n)$

→ This occurs when the distinct elements are arranged such that every single time you swap a leaf element to the root, it happens to be smaller than all its children.

Conclusion : The Max-Heap Sort algorithm was successfully implemented and executed to sort the elements in ascending order. The program verified a consistent time complexity of $O(n \log n)$ across the best, average, and worst cases using an in-place binary heap structure



Marwadi
University
Marwadi Chandarana Group



Faculty of Engineering and Technology
Department of CSE-AI / ML

01AI0506 - Design and Analysis of Algorithms